12. 아키텍처와 고급 기능

이 장에서는 Hermes Agent의 내부 아키텍처, 자기 진화 시스템, 그리고 고급 기능을 다룹니다. 에이전트를 사용하는 데 반드시 필요한 내용은 아니지만, 내부 작동을 이해하면 성능 튜닝, 문제 해결, 확장 개발에서 큰 도움이 됩니다.

이 장에서 다루는 내용

- 내부 구조
- 자기 진화 시스템 (DSPy + GEPA)
- 다중 에이전트 협업 (Kanban Multi-Agent)
- `hermes proxy` — OAuth 프로바이더용 로컬 프락시

12-1. 내부 구조

> ⚠️ ***부분 검증**: 이 페이지는 v0.16.0 소스 분석으로 확인했습니다. 에이전트 루프·트랜스포트·프롬프트 조립·캐싱·컨텍스트 압축·세션 영속화(SQLite/FTS5)·서브에이전트·플러그인 훅의 구조와 핵심 상수를 소스와 대조했습니다. 토큰 오버헤드 수치와 콜드 스타트 단축 폭은 부록의 실측·릴리스 노트 기반입니다. v0.14 이후 일부 코드가 모듈로 분리됐고 (루프는 `agent/conversation_loop.py` , 프롬프트 조립은 `agent/system_prompt.py`), 시스템 프롬프트는 stable·context·volatile 3계층으로 재구성됐습니다. 내부 구조 설명 위주라 별도 실행 테스트는 하지 않았습니다.*

- 내부 구조 개요
- 에이전트 루프 (run_agent.py)
  - ReAct 루프의 실제 구현
  - 이터레이션 버짓
  - 도구 실행 모드
  - Transport ABC와 4개 트랜스포트
- 매 턴 재전송 구조
- 프롬프트 조립 (agent/prompt_builder.py)
  - 시스템 프롬프트 3계층 구조
  - 캐시된 프롬프트 vs 에페메럴 레이어
  - 컨텍스트 파일 보안 검사
- 프롬프트 캐싱 (agent/prompt_caching.py)
  - system_and_3 전략
- 컨텍스트 압축 (agent/context_compressor.py)
  - 트리거 조건
  - 압축 알고리즘
  - 압축 후 세션 계보
- 세션 영속화 (hermes_state.py)
  - SQLite 기반 상태 저장
  - 데이터베이스 스키마
  - FTS5 전문 검색
  - WAL 모드와 동시성
- 서브에이전트 위임 (tools/delegate_tool.py)
  - 아키텍처
  - 실행 모드
  - 깊이 제한

- 플러그인 호스트 아키텍처
- 성능
  - Cold start
  - browser_console 영속 연결
- 버전별 변경
- 설계 원칙

내부 구조 개요

Hermes Agent는 모놀리식이 아닌, 느슨하게 결합된 서브시스템들로 구성되어 있습니다. 이 절에서는 소스 코드(`run_agent.py`, `agent/` 패키지, `hermes_state.py`)를 직접 분석하여 핵심 내부 작동을 설명합니다.

에이전트 루프 (`run_agent.py`)

ReAct 루프의 실제 구현

`AIAgent.run_conversation()` 메서드가 에이전트의 핵심 실행 루프입니다. 소스 코드에서 확인한 실제 흐름은 다음과 같습니다.

```
run_conversation(user_message)
  1. 이터레이션 버짓 초기화 (IterationBudget)
  2. 대화 히스토리 준비, todo 상태 복원
  3. 시스템 프롬프트 빌드 또는 캐시에서 로드
  4. 프리플라이트 압축 (필요시)
  5. while 루프 진입:
     a. 인터럽트 체크
     b. 이터레이션 버짓 소비
     c. API 메시지 조립 (에페메럴 레이어 주입)
     d. 프롬프트 캐싱 적용
     e. 고아 도구 페어 정리 (sanitize)
     f. API 호출 (스트리밍 또는 비스트리밍)
     g. 도구 호출이 있으면 → 실행 후 루프 계속
     h. 최종 텍스트면 → 세션 저장, 결과 반환
```

이터레이션 버짓

부모 에이전트와 서브에이전트가 **공유 이터레이션 버짓**을 사용합니다. `IterationBudget` 클래스가 스레드 안전한 카운터를 제공합니다.

```
# 소스 코드 (run_agent.py)에서 확인한 구조
class IterationBudget:
    def __init__(self, max_total: int):  # 기본값 90
        self.max_total = max_total
        self._used = 0
        self._lock = threading.Lock()

    def consume(self) -> bool:  # 1회 소비, 초과 시 False
    def refund(self) -> None:   # execute_code 턴은 환불
```

`execute_code` (프로그래매틱 도구 호출)로 소비된 이터레이션은 `refund()` 로 되돌려집니다. 이는 스크립트 실행이 실제 추론 턴이 아니기 때문입니다.

도구 실행 모드

Hermes는 두 가지 도구 실행 전략을 사용합니다.

순차 실행: 단일 도구 호출이거나, 인터랙티브 도구(`clarify` 등)가 포함된 경우

병렬 실행: 여러 도구가 동시에 요청되고, 안전 조건을 만족하는 경우. 소스 코드에서 확인한 안전 판단 로직은 다음과 같습니다.

- `_NEVER_PARALLEL_TOOLS` : `clarify` 처럼 절대 병렬 실행하지 않는 도구
- `_PARALLEL_SAFE_TOOLS` : `read_file` , `web_search` , `session_search` 등 읽기 전용 도구
- `_PATH_SCOPED_TOOLS` : `read_file` , `write_file` , `patch` — 대상 경로가 겹치지 않으면 병렬 허용
- 최대 동시 워커 수: 8개 (`_MAX_TOOL_WORKERS`)

Transport ABC와 4개 트랜스포트

API 호출 경로는 v0.11에서 **트랜스포트 추상화**(`agent/transports/`)로 분리됐습니다. 포맷 변환과 HTTP 실행이 한 클래스에서 책임지는 구조라, 새 프로바이더를 추가할 때 핵심 에이전트 루프를 건드리지 않습니다.

트랜스포트	`api_mode`	대상
`ChatCompletionsTransport`	`chat_completions`	OpenAI 호환 엔드포인트 (OpenRouter, Ollama, DeepSeek 등 16개+ 프로바이더)
`ResponsesApiTransport`	`codex_responses`	OpenAI Responses API (OpenAI 직접 연결, xAI)
`AnthropicTransport`	`anthropic_messages`	Anthropic Messages API (네이티브)
`BedrockTransport`	`bedrock_converse`	AWS Bedrock Converse API (네이티브)

OpenAI는 Chat Completions API(`/v1/chat/completions`)와 Responses API(`/v1/responses`)를 모두 제공합니다. Hermes는 OpenAI에 직접 연결할 때 Responses API를, OpenRouter 같은 호환 프로바이더를 경유할 때는 Chat Completions API를 사용합니다. 트랜스포트 선택은 아래의 자동 판별 로직이 처리합니다.

트랜스포트 선택은 대부분 자동입니다. `base_url` 의 호스트명을 보고 판별합니다.

- `api.openai.com` → `codex_responses`
- `api.x.ai` → `codex_responses`
- URL 끝이 `/anthropic` → `anthropic_messages`
- 그 외 → `chat_completions` (기본값)

`config.yaml` 의 `model.api_mode` 로 명시적으로 지정할 수도 있지만, 프로바이더를 바꿀 때 이전 프로바이더의 `api_mode` 가 남아 충돌하지 않도록 자동 무효화 로직이 들어 있습니다.

매 턴 재전송 구조

트랜스포트가 어느 것이든, Hermes가 사용하는 LLM API는 모두 **무상태(stateless)**입니다. 서버는 이전 대화를 기억하지 않으므로, Hermes Agent는 매 API 호출마다 다음을 모두 전송합니다.

1. **시스템 프롬프트**: 에이전트 정체성, 메모리, 스킬 인덱스, 컨텍스트 파일 등 (아래 프롬프트 조립 참조)
2. **도구 정의**: 등록된 모든 도구의 이름, 설명, 파라미터 스키마
3. **대화 이력 전체**: 첫 턴부터 현재까지의 모든 메시지

소스 코드(`run_agent.py`)에서 확인하면, `run_conversation()` 의 while 루프 안에서 매 이터레이션마다 `messages` 리스트 전체를 순회하여 `api_messages` 를 새로 구성하고, 시스템 프롬프트를 맨 앞에 삽입한 뒤 API에 전달합니다.

이 구조가 **토큰 오버헤드의 근본 원인**입니다. 부록 「로컬 LLM 가이드」에서 실측한 바와 같이, 도구 29개와 스킬 116개가 등록된 상태에서 단순한 질문 하나에도 약 14,000토큰이 소비됩니다. 턴이 쌓일수록 대화 이력도 함께 재전송되므로 토큰 소비는 가속됩니다.

이 비효율은 Hermes만의 문제가 아니라 무상태 LLM API를 사용하는 모든 에이전트 프레임워크의 공통 특성입니다. Claude API의 프롬프트 캐싱(아래 참조)은 시스템 프롬프트 부분의 실질 비용을 줄여주지만, 로컬 모델(Ollama 등)에서는 캐싱이 적용되지 않으므로 오버헤드가 그대로 컨텍스트 윈도우를 압박합니다.

프롬프트 조립 (`agent/prompt_builder.py`)

시스템 프롬프트 3계층 구조

v0.16 기준 `build_system_prompt_parts()` (`agent/system_prompt.py`)는 프롬프트를 세 계층으로 나눠 조립합니다. 캐시 친화 순서로 잘 변하지 않는 안정 계층을 앞에, 매 세션·턴 변동하는 휘발 계층을 뒤에 둡니다.

계층	포함 내용
stable (불변)	에이전트 아이덴티티(`SOUL.md` 우선, 없으면 `DEFAULT_AGENT_IDENTITY`) → Hermes 도움말 안내 → 작업 완수 가이던스 → 도구 기반 행동 가이던스(memory·session_search·skills 도구 로드 시) → 스킬 인덱스 → 실행 환경 힌트 → 플랫폼 힌트
context (cwd 의존)	호출자 제공 시스템 메시지 → 컨텍스트 파일(AGENTS.md, .cursorrules, .hermes.md)
volatile (변동·캐시 안 됨)	메모리 스냅숏(MEMORY.md) → 유저 프로필(USER.md) → 외부 메모리 프로바이더 블록 → 타임스탬프·모델 정보

예전의 평면 10단 구조에서 바뀐 점은, 스킬 인덱스·플랫폼 힌트가 안정 계층으로 올라가고 MEMORY·USER가 휘발 계층(맨 뒤)으로 내려간 것입니다. Honcho 전용 레이어는 일반 외부 메모리 프로바이더 블록으로 일반화됐습니다.

캐시된 프롬프트 vs 에페메럴 레이어

Hermes의 프롬프트 설계에서 가장 중요한 아키텍처 결정은 **캐시되는 부분과 일시적인 부분의 분리**입니다.

캐시 대상 (세션 DB에 저장, 세션 내내 불변):

- 위 stable·context·volatile 세 계층을 합친 시스템 프롬프트 전체(한 덩어리로 캐시)

에페메럴 (API 호출 시에만 주입, 저장 안 됨):

- `ephemeral_system_prompt` (서브에이전트 태스크 프롬프트 등)
- `prefill_messages` (few-shot 프라이밍)
- Honcho 후속 턴 리콜 (현재 턴 user 메시지에 주입)

소스 코드에서 명시적으로 확인되는 패턴입니다.

```
# run_conversation()에서 발췌한 흐름
if self._cached_system_prompt is None:
    if stored_prompt:  # 기존 세션이면 DB에서 로드
        self._cached_system_prompt = stored_prompt
    else:              # 새 세션이면 빌드
        self._cached_system_prompt = self._build_system_prompt(system_message)

# API 호출 시에만 에페메럴 추가
effective_system = active_system_prompt
if self.ephemeral_system_prompt:
    effective_system = effective_system + "\n\n" + self.ephemeral_system_prompt
```

이 분리의 목적은 **Anthropic 프롬프트 캐싱의 히트율을 극대화**하는 것입니다. 시스템 프롬프트가 매 턴 변하면 캐시가 무효화되어 비용이 크게 증가합니다.

컨텍스트 파일 보안 검사

`prompt_builder.py`는 AGENTS.md, .cursorrules 등 외부 컨텍스트 파일을 주입하기 전에 **프롬프트 인젝션 스캔**을 수행합니다. 소스 코드에서 확인한 탐지 패턴의 예시입니다.

- `ignore previous instructions` 같은 프롬프트 인젝션 패턴
- 숨겨진 HTML (`display:none`)
- 비가시 유니코드 문자 (zero-width space 등)
- `curl`을 통한 시크릿 유출 시도
- `.env` , `.netrc` 등 민감 파일 읽기 시도

탐지되면 해당 파일 내용이 차단되고 `[BLOCKED: ...]` 메시지로 대체됩니다.

프롬프트 캐싱 (`agent/prompt_caching.py`)

system_and_3 전략

소스 코드에서 확인한 캐싱 전략입니다. Anthropic의 `cache_control` 마커를 최대 4개 배치합니다.

브레이크포인트	대상
1	시스템 프롬프트
2~4	마지막 3개 비시스템 메시지 (롤링 윈도우)

```
# prompt_caching.py에서 확인한 구조
marker = {"type": "ephemeral"}  # 기본 TTL: 5분
if cache_ttl == "1h":
    marker["ttl"] = "1h"
```

이 전략은 Anthropic 공식 문서의 브레이크포인트 상한(4개)을 정확히 활용합니다. 시스템 프롬프트에 1개, 나머지 3개를 최근 메시지에 배치하여 프리픽스 캐시 히트를 극대화합니다.

OpenRouter 경유 시에도 캐싱 자동 적용

OpenRouter를 통해 Claude 모델을 사용할 때도 이 캐싱이 자동 적용됩니다. 모델 이름에 "claude"가 포함되어 있으면 `_use_prompt_caching`이 활성화됩니다.

컨텍스트 압축 (`agent/context_compressor.py`)

트리거 조건

압축은 두 시점에서 발생할 수 있습니다.

1. **프리플라이트 압축**: `run_conversation()` 진입 시, 기존 대화 히스토리가 이미 임계값을 초과하는 경우 (예: 더 작은 컨텍스트 윈도우 모델로 전환 시)
2. **인라인 압축**: API 응답의 실제 토큰 카운트가 임계값을 초과하는 경우

임계값은 기본적으로 **모델 컨텍스트 길이의 50%**입니다.

```
# context_compressor.py에서 확인
threshold_percent = 0.50  # 기본값
protect_first_n = 3       # 보호할 앞부분 턴 수
protect_last_n = 20       # 보호할 뒷부분 턴 수
summary_target_ratio = 0.20   # 요약 목표 비율 (임계 토큰 대비)
```

압축 알고리즘

소스 코드에서 확인한 압축 절차입니다.

1. 앞의 3턴과 뒤의 20턴을 **보호 영역**으로 지정합니다
2. 보호 영역 사이의 **중간 턴**들을 요약 대상으로 선택합니다
3. 경계 조정: 도구 호출/결과 쌍이 분리되지 않도록 경계를 앞뒤로 밀어냅니다
4. 보조 LLM(`call_llm`의 `compression` 태스크)으로 중간 턴을 요약합니다
5. 요약을 `[CONTEXT COMPACTION]` 프리픽스와 함께 단일 메시지로 삽입합니다

6. **고아 도구 페어 정리**: 요약으로 잘린 tool_call/tool_result 쌍을 수복합니다

고아 페어 정리는 두 가지 경우를 처리합니다.

- **고아 결과**: tool_call이 요약에 포함되어 사라졌는데 result만 남은 경우 → 해당 result 삭제
- **고아 호출**: tool_call은 남았는데 result가 요약에 포함되어 사라진 경우 → 스텁 result 삽입

압축 후 세션 계보

압축이 발생하면 **새 세션 ID**로 분할될 수 있으며, `parent_session_id`로 원래 세션과의 계보가 유지됩니다. 이로써 검색 시 세션 가족 단위로 추적할 수 있습니다.

세션 영속화 (`hermes_state.py`)

SQLite 기반 상태 저장

소스 코드에서 확인한 저장소는 **SQLite** (`~/.hermes/state.db`)이며, JSONL이 아닙니다.

> SQLite가 주 저장소, JSONL은 레거시
>
> 공식 문서에도 "some legacy JSON/JSONL artifacts still exist for compatibility, but SQLite is the main historical store"라고 명시되어 있습니다. 즉 SQLite가 주 저장소이고, 레거시 JSONL은 호환용으로만 남아 있습니다.

데이터베이스 스키마

소스 코드에서 확인한 핵심 테이블 구조입니다 (스키마 버전 14).

sessions 테이블:

- `id` : 세션 고유 ID (TEXT PRIMARY KEY)
- `source` : 플랫폼 (cli, telegram, discord 등)
- `parent_session_id` : 압축 분할 시 부모 세션 참조
- `model` , `system_prompt` : 사용 모델 및 시스템 프롬프트 스냅숏
- `title` : 세션 제목 (고유, 100자 제한)
- 토큰 카운트: `input_tokens` , `output_tokens` , `cache_read_tokens` , `cache_write_tokens` , `reasoning_tokens`
- 비용 추적: `estimated_cost_usd` , `actual_cost_usd` , `billing_provider`

messages 테이블:

- `session_id` , `role` , `content` , `tool_call_id` , `tool_calls` , `tool_name` , `timestamp`

FTS5 전문 검색

`messages_fts` 라는 FTS5 가상 테이블이 메시지 내용의 전문 검색을 제공합니다. 트리거로 메시지 삽입/삭제/수정 시 자동 동기화됩니다. `session_search` 도구가 이 인덱스를 사용합니다.

검색 문법은 FTS5 표준을 따릅니다. 키워드, `"정확한 구문"`, `OR` / `NOT` 불리언, `접두사*` 등이 지원됩니다. 하이픈이 포함된 용어(예: `chat-send`)는 자동으로 따옴표로 감싸서 구문 매칭됩니다.

WAL 모드와 동시성

데이터베이스는 **WAL(Write-Ahead Logging) 모드**로 작동합니다. 게이트웨이 환경에서 여러 읽기 스레드와 하나의 쓰기 스레드가 동시에 작동할 수 있게 합니다. 스레드 잠금(`threading.Lock`)으로 쓰기 작업의 직렬화를 보장합니다.

서브에이전트 위임 (`tools/delegate_tool.py`)

아키텍처

`delegate_task` 도구는 자식 `AIAgent` 인스턴스를 생성하여 독립된 컨텍스트에서 작업을 수행합니다.

자식 에이전트의 특징:

- **격리된 대화**: 부모의 히스토리를 상속하지 않습니다
- **독립된 터미널 세션**: 별도의 작업 디렉터리와 상태
- **제한된 도구셋**: `delegate_task`, `clarify`, `memory`, `send_message`, `execute_code`는 차단됩니다
- **공유 이터레이션 버짓**: 부모와 동일한 `IterationBudget` 인스턴스를 사용합니다

실행 모드

- **단일 태스크**: `goal`만 제공하면 직접 실행 (스레드 풀 오버헤드 없음)
- **배치 (병렬)**: `tasks` 배열로 최대 3개를 `ThreadPoolExecutor`로 병렬 실행

깊이 제한

소스 코드에서 확인한 재귀 제한입니다.

```
MAX_DEPTH = 1          # 기본은 평면: parent(0) -> child(1), grandchild 거부
MAX_CONCURRENT_CHILDREN = 3
DEFAULT_MAX_ITERATIONS = 50  # 자식 에이전트 기본 이터레이션
```

서브에이전트는 기본적으로 다른 서브에이전트를 생성할 수 없습니다(설정 `max_spawn_depth`를 높이지 않는 한 grandchild는 거부됩니다).

플러그인 호스트 아키텍처

v0.11~v0.12에서 게이트웨이가 **플러그인 호스트**로 정착했습니다. 메시징 플랫폼 어댑터, 옵저버빌리티 플러그인, 컨텍스트 엔진 — 모두 같은 플러그인 슬롯을 통해 코어 외부에서 결합됩니다.

플러그인이 끼워 넣을 수 있는 주요 훅:

훅	시점	용도
`on_session_start` / `on_session_finalize` / `on_session_reset`	세션 라이프사이클	세션별 자원 정리, 외부 트래킹 시스템 동기화
`pre_tool_call` (veto 가능)	도구 호출 직전	도구 호출을 거부하거나 통과
`post_tool_call` (`duration_ms` 포함)	도구 호출 직후	메트릭, 결과 사후 처리
`transform_tool_result`	결과를 모델에 돌려주기 직전	결과 가공·필터링
`transform_terminal_output`	터미널 출력 직전	출력 가공·요약
`pre_gateway_dispatch`	메시지가 게이트웨이로 들어오는 시점	라우팅·필터링
`pre_approval_request` / `post_approval_response`	위험 명령 승인 전후	외부 감사·SOC 통합

또한 **컨텍스트 엔진**이 플러그형 슬롯이 됐습니다. `hermes plugins` 로 등록한 사용자 정의 컨텍스트 엔진이 매 턴 모델에 전달할 컨텍스트를 가공할 수 있습니다 — 도메인 특화 인젝션·필터링·요약을 코어 변경 없이 끼워 넣을 수 있습니다.

성능

내부 구조 변경으로 가시 응답 속도가 크게 줄어든 두 항목을 한 곳에 모았습니다. 버전별 변경 시점은 다음 「버전별 변경」 절에서 함께 정리합니다.

Cold start

게이트웨이·CLI 진입 시점의 가시 cold start가 누적적으로 줄었습니다. 두 단계 변경이 있습니다.

- **모듈 lazy import** — OpenAI·Anthropic·Firecrawl·account_usage 모듈을 시작 시점이 아닌 첫 사용 시점에 로드
- **설정·도구 정의 캐시** — `load_config()` · `read_raw_config()` 을 mtime 기반으로 캐시, `get_tool_definitions()` 메모이즈, `check_fn` 결과 TTL 캐시

- **정규식 사전 컴파일** — `DANGEROUS_PATTERNS` · `HARDLINE_PATTERNS` 정규식을 모듈 로드 시 한 번만 컴파일
- **스킬 카탈로그·HTTP 호출 정리** — 스킬 캐시, Feishu lazy 로드, Nous HTTP 호출 제거, 모델 카탈로그 디스크 캐시, doctor 검사 병렬화, `chat -q` 에서 환영 배너 생략

부가 효과로 `hermes -z` 일회성 모드의 응답 시작도 같이 빨라지고, `hermes tools` All-Platforms 화면이 14초에서 1.5초 미만으로 떨어졌습니다.

`browser_console` 영속 연결

브라우저 도구가 페이지를 검사하거나 JavaScript를 실행할 때, 매번 새 DevTools 세션을 띄우는 대신 **하나의 영속 Chrome 연결**을 공유합니다. 호출당 수 초 걸리던 작업이 밀리초 단위로 회신되며, 실제 페이지 조작이 즉각 반응합니다.

버전별 변경

내부 구조와 관련된 변경 시점을 한 곳에 모았습니다. 자세한 내용은 위 본문의 해당 절에서 다룹니다.

- **v0.14** — Cold start 추가 단축(~19초). `browser_console` 이 영속 Chrome 연결을 공유하며 ~180× 속도 향상.
- **v0.12** — Cold start가 약 57% 줄어듦(모듈 lazy import·설정 캐시·정규식 사전 컴파일). 게이트웨이 플러그인 호스트 정착.
- **v0.11** — 트랜스포트 추상화(Transport ABC) 도입으로 4개 트랜스포트가 분리. 게이트웨이가 플러그인 호스트로 발전 시작.

설계 원칙

소스 코드 분석과 공식 개발자 가이드에서 일관되게 확인되는 설계 원칙입니다.

1. **프롬프트 안정성**: 시스템 프롬프트를 세션 내에서 불변으로 유지하여 캐시 히트율을 극대화합니다
2. **관찰 가능하고 중단 가능한 도구 실행**: 콜백 시스템(`tool_progress_callback` , `step_callback` 등)으로 모든 도구 호출을 추적하고, 사용자가 새 메시지를 보내면 루프를 중단할 수 있습니다
3. **영속 세션과 계보**: SQLite에 세션과 메시지를 저장하고, 압축 분할 시에도 `parent_session_id` 로 계보를 유지합니다
4. **통합 에이전트 코어**: CLI, 게이트웨이, 크론, ACP 모든 플랫폼에서 동일한 `AIAgent` 클래스를 사용합니다
5. **느슨한 결합**: Honcho, 스킬, 메모리, 컨텍스트 엔진, 메시징 플랫폼 어댑터 등 선택적 서브시스템이 독립적으로 활성화/비활성화됩니다 — 그 부분을 플러그인이 채울 수 있습니다

12-2. 자기 진화 시스템 (DSPy + GEPA)

> ⚠️ ***부분 검증**: v0.16.0 메인 저장소 소스 분석 기준입니다(GEPA·self-evolution은 메인 저장소에 없음, dspy는 옵션 스킬). 자기 진화 시스템은 별도 저장소라 이 책에서 직접 구동·검증하지는 않았습니다.*

- 자기 진화 시스템
- 메인 저장소와의 관계
- hermes-agent-self-evolution
- 최적화 대상
- 작동 방식
- 비용과 요구사항
- GEPA의 출처
- 메인 저장소에서의 자기 개선 메커니즘
  - 스킬 자동 생성/수정 + 큐레이터
  - 메모리 자동 저장
  - 궤적 저장과 RL 파이프라인
- 일반 사용자에게의 의미

자기 진화 시스템

Hermes Agent의 자기 개선은 사용 중의 스킬 생성/수정에 그치지 않습니다. NousResearch는 별도의 **자기 진화(self-evolution)** 시스템을 개발했습니다.

메인 저장소와의 관계

tip

소스 코드 분석 결과: 자기 진화 시스템(DSPy + GEPA)은 메인 `hermes-agent` 저장소에 포함되어 있지 않습니다. 메인 저장소에서 "GEPA"나 "self-evolution"을 검색하면 관련 코드가 전혀 나오지 않습니다. DSPy는 `optional-skills/mlops/research/dspy/` 경로에 **옵션 참조 스킬** 형태로만 존재하며, 이는 DSPy 프레임워크 사용법을 안내하는 일반 스킬일 뿐 에이전트 자기 진화 기능과는 무관합니다.

자기 진화 시스템은 GitHub 저장소 NousResearch/hermes-agent-self-evolution에서 **별도 프로젝트**로 관리됩니다. 일상적인 Hermes Agent 사용에는 이 별도 저장소가 필요하지 않습니다.

hermes-agent-self-evolution

이 별도 시스템은 **DSPy + GEPA(Genetic-Pareto Prompt Evolution)**를 사용하여 에이전트의 구성요소를 자동으로 최적화합니다.

최적화 대상

- **스킬**: 더 효과적인 절차로 진화
- **도구 설명**: LLM이 도구를 더 정확하게 선택하도록 설명 개선
- **시스템 프롬프트**: 더 효과적인 프롬프트로 진화
- **코드**: 에이전트의 코드 자체를 개선

작동 방식

1. 현재 스킬/프롬프트/코드의 성능을 측정합니다
2. 변형(mutation)을 생성합니다
3. 각 변형의 성능을 평가합니다
4. 파레토 최적(Pareto optimal) 변형을 선택합니다
5. 다음 세대로 진행합니다

이 과정을 반복하면 측정 가능하게 더 나은 변형이 산출됩니다.

비용과 요구사항

- API 호출 기반이므로 GPU 학습이 필요 없습니다
- 최적화 한 번에 약 \$2~\$10 정도 소요됩니다
- 모델의 가중치를 변경하지 않습니다 (프롬프트/스킬 수준 최적화)

GEPA의 출처

`hermes-agent-self-evolution` 저장소의 README에는 "ICLR 2026 Oral"이라는 문구가 있습니다. 이것은 Hermes Agent 자체가 학회에 발표되었다는 뜻이 아닙니다.

GEPA(Genetic-Pareto)는 Stanford DSPy 팀이 개발한 **독립적인 프롬프트 최적화 기법**입니다. 논문 정보는 다음과 같습니다.

- **논문**: "GEPA: Reflective Prompt Evolution Can Outperform Reinforcement Learning"
- **저자**: Lakshya A Agrawal, Shangyin Tan 외 15명 (Stanford, Berkeley, UT Austin 등)
- **학회**: ICLR 2026 Oral
- **arXiv**: https://arxiv.org/abs/2507.19457
- **코드**: https://github.com/gepa-ai/gepa

논문의 핵심 결과는, GEPA가 GRPO 대비 평균 6%(최대 20%) 높은 성능을 35배 적은 롤아웃으로 달성했다는 것입니다. 에이전트의 실행 궤적(추론, 도구 호출, 도구 출력)을 자연어로 반성(reflect)하여 프롬프트 개선점을 진단하는 방식입니다.

정리하면, Hermes Agent의 자기 진화 시스템은 이 GEPA 논문의 기법을 **활용하는** 관계입니다. NousResearch가 GEPA를 만든 것이 아니라, 학회에서 검증된 외부 기법을 에이전트 최적화에 적용한 것입니다.

메인 저장소에서의 자기 개선 메커니즘

메인 저장소에서 확인할 수 있는 자기 개선 관련 기능은 다음과 같습니다.

스킬 자동 생성/수정 + 큐레이터

에이전트는 복잡한 작업(도구를 여러 번 호출하는 작업)을 완료하면 그 절차를 스킬로 저장하도록 유도됩니다. 기존 스킬이 오래되었거나 불완전하면 즉시 수정합니다. 또한 도구 호출이 일정 횟수 누적되면 백그라운드에서 리뷰 에이전트가 스킬 생성 여부를 검토합니다 (7-2절 참조).

v0.12에서는 이 위에 **백그라운드 큐레이터**가 추가됐습니다. 7일에 한 번씩 라이브러리 전체를 리뷰해서 비슷한 스킬을 합치고(consolidated), 오랫동안 쓰이지 않은 스킬을 솎아냅니다(pruned). 자세한 내용은 7-6절 참고.

리뷰 포크 자체도 v0.12에서 정비됐습니다.

- 자유 형식 판단 대신 **클래스 우선 루브릭**으로 등급을 매김
- 직전에 로드된 스킬을 **우선 갱신** 대상으로 선택(active-update bias)
- `references/` · `templates/` 하위 파일까지 갱신 범위에 포함
- 리뷰 포크가 부모 대화의 프로바이더·모델·자격증명을 그대로 **상속**(이전에는 일부 환경에서 다른 모델로 폴백되어 품질이 흔들렸음)
- 리뷰 포크는 `memory` · `skills` 도구 셋만 활성화 — 셸·웹·터미널로 새지 않음
- 리뷰가 보는 컨텍스트에는 메인 대화의 **이전 턴 도구 메시지가 빠짐** → 잡음 없는 평가

메모리 자동 저장

일정 턴 수가 지나면 백그라운드 리뷰 에이전트가 메모리 저장 여부를 검토합니다. 다만 이것은 사용자 선호도와 환경 정보를 기억하는 것이지, 에이전트의 행동 자체를 최적화하는 메커니즘은 아닙니다.

궤적 저장과 RL 파이프라인

메인 저장소에는 에이전트의 작업 궤적을 학습 데이터로 변환하는 도구와, Atropos 기반 RL 학습 인터페이스가 포함되어 있습니다. 궤적 압축은 완료된 대화를 후처리하여 토큰 예산 내로 줄이는 것으로, 대화 중에 작동하는 컨텍스트 압축(5장 참조)과는 별개의 시스템입니다. 이들은 에이전트 런타임의 일부가 아니라 별도의 학습 워크플로우입니다.

일반 사용자에게의 의미

대부분의 사용자는 자기 진화 시스템을 직접 사용할 필요가 없습니다. 일상적인 사용에서는 에이전트의 내장 스킬 자동 생성/수정 메커니즘(7장 참조)으로 충분합니다.

하지만 이 시스템의 존재는 Hermes Agent가 단순한 엔지니어링 프로젝트가 아니라, **연구 기반의 프레임워크**임을 보여줍니다. RL 학습 파이프라인(Atropos)을 자체 구축하고, 학회에서 검증된 외부 기법(GEPA)을 프롬프트 최적화에 적용하는 등 여러 방향의 자기 개선을 시도하고 있습니다.

12-3. Kanban Multi-Agent 협업

> ⚠️ ***부분 검증**: 아래 내용은 v0.16.0 소스 코드(`tools/kanban_tools.py` , `hermes_cli/kanban.py` · `kanban_db.py` · `kanban_specify.py` · `kanban_decompose.py` · `kanban_diagnostics.py`)를 분석한 결과입니다. 카드 상태는 v0.16.0 기준 9가지이며, 본문은 핵심 흐름만 다룹니다. 실제 운영 테스트는 아직 진행하지 않았습니다.*

- 전체 흐름
- 카드 상태 흐름
- 주요 도구
- specify — 작업 분해 보조 LLM
- 안전장치
- 언제 쓰면 좋은가

같은 머신에서 여러 Hermes 에이전트가 협업하는 구조가 v0.14.0에서 본격적으로 도입됐습니다. 한 프로필이 작업을 잘게 쪼개 보드에 올리면, 다른 프로필이 그 작업을 가져가 처리하고, 또 다른 프로필이 결과를 평가·전달합니다. 보드가 동기화 지점 역할을 하므로, 에이전트끼리 직접 공유해야 하는 컨텍스트가 줄어듭니다.

전체 흐름

```
flowchart TD
    A["오케스트레이터\n(specify로 분해)"] -- "todo / ready" --> B["Kanban 보드\n(SQLite 영속화)"]
    B -- "claim" --> C["워커 #1\n(도구 실행)"]
    B -- "claim" --> D["워커 #N\n(도구 실행)"]
    C -- "done" --> E["Notifier\n(메시지·이벤트 발송)"]
    D -- "done" --> E
```

세 역할은 모두 일반 Hermes 에이전트입니다. 다만 `config.yaml` 의 프로필 설정에 따라 사용하는 도구와 모델만 다릅니다. 보드는 SQLite에 저장되므로, 어느 한 프로필의 실행이 중단돼도 다른 프로필이 이어받을 수 있습니다.

카드 상태 흐름

카드는 다음 상태를 거칩니다.

v0.16.0 기준 카드 상태는 `triage` · `todo` · `scheduled` · `ready` · `running` · `blocked` · `review` · `done` · `archived` 9가지입니다. 핵심 흐름은 다음과 같습니다.

```
flowchart LR
    triage --> todo --> ready --> running --> done
    running -- "검토 요청" --> review --> running
    running -- "블로커 발생" --> blocked -- "해제" --> running
    ready -- "오래 머무름" --> stranded_in_ready
```

상태	의미
`triage`	러프한 아이디어(보통 제목만) 상태. `specify` / `decompose` 가 사양을 채워 `todo` 로 승격
`todo`	사양이 확정돼 의존 카드 완료를 기다리는 상태. 의존이 없으면 곧 `ready` 로 승격
`ready`	워커가 가져갈 수 있는 상태. 의존 카드가 모두 `done` (또는 `archived`)이 되면 자동 승격
`running`	워커가 클레임한 상태. 클레임은 기본 약 15분간 유효하며 heartbeat로 갱신됩니다. 갱신 없이 만료되면 다음 디스패처 틱이 회수하고, 대시보드는 `reclaim` · `reassign` 을 권고합니다
`blocked`	`kanban_block` 으로 표시한 상태. `kanban_unblock` (오케스트레이터 전용)으로 풀림
`review`	워커가 사람 검토를 요청하며 옮긴 상태. 검토자가 다시 `running` 으로 클레임
`done`	워커가 `kanban_complete` 로 마친 상태
`stranded_in_ready`	진단상 상태. 워커가 오랜 시간 클레임하지 않은 `ready` 카드

상태 전이는 모두 SQLite 트랜잭션으로 처리되며, 두 워커가 같은 카드를 동시에 잡으려 해도 한쪽만 성공합니다(아래 「안전장치」 참고).

주요 도구

오케스트레이터·워커가 호출하는 도구는 `tools/kanban_tools.py` 에 모여 있습니다.

도구	역할
`kanban_create`	새 작업 카드 생성. 의존(`parents`)이 있으면 `todo` , 없으면 `ready` 로 승격. `triage=True` 면 `triage` 로 진입
`kanban_list`	보드 상태별·소유자별 카드 조회 (오케스트레이터 전용)
`kanban_show`	단일 카드 상세 조회
`kanban_complete`	완료 처리 (run id로 소유권 검증)
`kanban_block`	블로커 등록·블록 사유 기록
`kanban_unblock`	블로커 해제 (오케스트레이터 전용)
`kanban_heartbeat`	워커가 살아 있음을 보드에 통지
`kanban_comment`	카드에 코멘트 추가
`kanban_link`	카드 간 의존 관계 등록

specify — 작업 분해 보조 LLM

오케스트레이터가 큰 목표를 받으면 보조 LLM이 그 목표를 보드에 올릴 만한 단위로 쪼개 줍니다. `specify` (`hermes_cli/kanban_specify.py`)는 카드 하나의 사양을 다듬어 `triage` 에서 `todo` 로 올리고, 여러 카드로 펼치는 그래프 분해는 `hermes kanban decompose` (`hermes_cli/kanban_decompose.py`)가 맡습니다. 보조 LLM이 작업 분해와 우선순위 결정을 맡으므로, 주 모델의 컨텍스트 부담이 줄어듭니다.

안전장치

여러 프로필이 같은 보드를 함께 사용하므로, v0.14 소스 코드에는 동시성과 복구 상황에 대비한 안전장치가 다음과 같이 들어 있습니다. 실제 운영 테스트는 후속 작업으로 진행 예정입니다.

- **원자적 클레임(atomic claim)**: 카드를 `ready` 에서 `running` 으로 옮길 때 단일 SQLite 트랜잭션으로 처리합니다. 코드상 두 워커가 같은 카드를 동시에 잡으려 해도 한쪽만 성공하고, 다른 쪽은 즉시 실패하도록 작성돼 있습니다.
- `stranded_in_ready` **진단**: `hermes_cli/kanban_diagnostics.py` 가 `ready` 상태로 오래 머무는 카드를 감지합니다. 워커가 중단됐거나 heartbeat를 보내지 못한 채 멈춘 카드를 표시하므로, 오케스트레이터가 해당 카드를 회수하거나 다시 할당할 수 있습니다.
- **notifier rewind**: 메시지를 보낸 뒤 응답을 받지 못해 재시도할 때, notifier가 같은 메시지를 두 번 보내지 않도록 보드의 마지막 성공 시점으로 되돌아갑니다.

- `kanban_comment` author **검증 강화**: 코멘트 작성자를 프로필 식별자에서 직접 가져오도록 작성돼, 다른 워커 이름으로 코멘트를 남기는 우회 경로를 차단합니다.

언제 쓰면 좋은가

- **작업이 독립 조각으로 잘 쪼개지는 경우**: 코드 변경 N개, 페이지 N개 정정, 외부 호출 N건 같은 패턴
- **다른 모델·도구를 섞고 싶을 때**: 오케스트레이터에는 추론에 강한 모델을, 워커에는 빠르고 저렴한 모델을 두어 비용과 속도의 균형을 맞춥니다.
- **무인 운영**: `/goal`과 결합해 모든 카드가 완료될 때까지 자동으로 진행합니다.
- **사람-에이전트 혼합**: 사람이 카드를 추가하고 에이전트가 처리, 또는 그 반대

반대로 **단일 작업·짧은 대화·실시간 사람 입력 중심**의 흐름에서는 보드를 관리하는 부담이 더 큽니다. 이런 경우에는 일반적인 단일 세션으로 끝내는 편이 자연스럽습니다.

실전 사례는 13장에서

13장의 실전 프로젝트(코드 분석·뉴스 브리핑·팀 봇 등) 중 작업이 충분히 잘게 나뉘는 경우는 Kanban Multi-Agent 구성으로 옮길 수 있습니다. 13장에서는 실제 보드 운영 예시와 모델 슬롯 배분 방식을 다룹니다.

12-4. hermes proxy — OAuth 프로바이더용 로컬 프록시

`hermes proxy`는 OAuth로 로그인하는 프로바이더를 Aider나 Open WebUI 같은 외부 도구에서 쓸 수 있게 해 주는 로컬 프락시입니다.

> ⚠️ ***부분 검증**: 이 페이지는 v0.14.0 기준입니다. 공식 가이드와 소스 코드를 바탕으로 정리했지만, 아직 직접 실행해 보지는 않았습니다.*

- 프락시 시작하기
- 활용 시나리오
- 작동 방식

프락시 시작하기

프로바이더에 따라 OAuth 로그인만 제공하고 별도 API 키를 발급하지 않는 경우가 있습니다. 이런 프로바이더는 API 키를 요구하는 외부 도구에서 바로 쓸 수 없는데, `hermes proxy`가 그 사이를 연결합니다.

v0.14.0에는 Nous Portal 어댑터만 포함되어 있습니다. Nous Portal은 Hermes Agent를 만든 Nous Research의 추론 서비스입니다. 어댑터 인터페이스(`UpstreamAdapter`)가 공개되어 있어 다른 OAuth 프로바이더도 추가할 수 있는 구조이지만, SuperGrok 등 다른 어댑터는 아직 없습니다.

먼저 `hermes login nous`로 Nous Portal에 로그인합니다. 명령을 실행하면 브라우저가 열리고, 안내에 따라 인증하면 됩니다.

```
hermes login nous                       # Nous Portal OAuth 로그인
```

로그인을 마친 뒤 프락시를 시작합니다.

```
hermes proxy start                      # 기본 포트 8645에서 시작
hermes proxy start --port 8080          # 포트 바꾸기
hermes proxy start --host 0.0.0.0       # 같은 LAN의 다른 기기에서도 접근 허용
hermes proxy status                     # 프로바이더 연결 상태 확인
hermes proxy providers                  # 프락시로 연결할 수 있는 프로바이더 목록
```

> ⚠️ ***LAN 노출 시 주의:*** *`--host 0.0.0.0` 으로 열면 같은 네트워크의 다른 사람이 이 프락시를 통해 내 계정으로 요청을 보낼 수 있습니다. 프락시 자체에는 별도 인증이 없으므로, 신뢰할 수 있는 네트워크에서만 사용하세요.*

`hermes proxy providers` 로 현재 버전에서 지원하는 프로바이더를 확인할 수 있습니다. `--provider` 옵션으로 프로바이더를 지정할 수 있으며, 생략하면 기본값인 `nous` 가 선택됩니다.

활용 시나리오

OpenAI 호환 API를 지원하는 외부 도구에서 API Base URL을 프락시 주소(`http://localhost:8645/v1`)로 지정하면 됩니다. API 키 입력란에는 `dummy` 같은 임의 문자열을 넣습니다. 프락시가 OAuth 토큰을 대신 붙여 주므로 이 값은 실제 인증에 쓰이지 않습니다.

```
# 예: Aider에서 hermes proxy 경유로 사용
# .aider.conf.yml
openai-api-base: http://localhost:8645/v1
openai-api-key: dummy
model: <모델명>          # hermes proxy providers 또는 /v1/models에서 확인
```

```
# 예: 환경변수로 설정
# 도구에 따라 OPENAI_API_BASE 대신 OPENAI_BASE_URL을 요구할 수 있습니다.
export OPENAI_API_BASE=http://localhost:8645/v1
export OPENAI_API_KEY=dummy
```

작동 방식

프락시는 Hermes Agent의 대화 흐름을 거치지 않습니다. 외부 도구의 요청에 OAuth 토큰만 붙여서 프로바이더의 추론 서버로 그대로 전달합니다.

프락시가 받아 주는 API 경로는 다음과 같습니다.

경로	용도
`/v1/chat/completions`	채팅 (스트리밍 포함)
`/v1/completions`	텍스트 완성
`/v1/embeddings`	임베딩
`/v1/models`	모델 목록

그 밖의 경로(`/v1/images/generations`, `/v1/audio/speech` 등)로 요청하면 404로 응답합니다. X 검색, TTS, 이미지 생성 같은 Hermes Agent의 도구는 에이전트 안에서만 실행되며, 이 프락시를 통해 사용할 수 없습니다.